

Nx Microfrontend Architecture with Module Federation

Production Deployment Blueprint for AKS (Shell + Employee Remote)

1. Overview of the Architecture

This architecture uses Nx + Angular + Module Federation to create a scalable microfrontend platform where:

- **Shell** is the host application deployed as an NGINX pod on AKS.
- **Employee** is a remote microfrontend deployed independently as its own NGINX pod.
- The shell loads the employee remote **lazily at runtime** using Module Federation.
- Both apps are built, versioned, and deployed independently.
- AKS provides isolation, scalability, and independent release cycles.

This pattern supports large enterprise Angular platforms with multiple teams and domains.

2. Nx Workspace Structure

apps/ shell/ → Host (Module Federation) employee/ → Remote (Module Federation)
libs/ shared-ui/ shared-utils/ shared-data-access/

- Apps are deployable microfrontends.
- Libraries contain shared logic but are not part of the MFE boundary.
- Remotes are full Angular apps, not libraries.

3. Module Federation Configuration

Shell (Host)

```
apps/shell/module-federation.config.js
```

```
module.exports = { name: 'shell', remotes: [ ['employee',  
process.env.EMPLOYEE_REMOTE_URL], ], };
```

Employee (Remote)

```
apps/employee/module-federation.config.js
```

```
module.exports = { name: 'employee', exposes: { './Module':  
'apps/employee/src/app/remote-entry/entry.module.ts', }, };
```

4. Lazy Loading the Employee Remote

The shell loads the employee remote only when the user navigates to `/employee`.

`shell/src/app/app.routes.ts`

```
import { loadRemoteModule } from '@nx/module-federation'; export const appRoutes: Routes = [ { path: 'employee', loadChildren: () => loadRemoteModule('employee', './Module').then(m => m.RemoteEntryModule), }, ];
```

This ensures:

- No employee code is bundled into the shell.
- The remote is fetched via HTTP at runtime.
- The shell and employee remain independently deployable.

5. Build Commands

Shell

```
nx build shell --configuration=production
```

Employee

```
nx build employee --configuration=production
```

Output:

```
dist/apps/shell/ dist/apps/employee/
```

6. Dockerfiles

Shell Dockerfile

```
FROM node:20-alpine AS builder WORKDIR /app COPY . . RUN npm install RUN npx nx build shell --configuration=production FROM nginx:1.25-alpine COPY dist/apps/shell/ /usr/share/nginx/html EXPOSE 80 CMD ["nginx", "-g", "daemon off;"]
```

Employee Dockerfile

```
FROM node:20-alpine AS builder WORKDIR /app COPY . . RUN npm install RUN npx nx build employee --configuration=production FROM nginx:1.25-alpine COPY dist/apps/employee/ /usr/share/nginx/html EXPOSE 80 CMD ["nginx", "-g", "daemon off;"]
```

7. AKS Deployment Blueprint

7.1 Shell Deployment

`shell-deployment.yaml`

```
apiVersion: apps/v1 kind: Deployment metadata: name: shell spec: replicas: 3 selector:
matchLabels: app: shell template: metadata: labels: app: shell spec: containers: - name:
shell image: .azurecr.io/shell: ports: - containerPort: 80 env: - name:
EMPLOYEE_REMOTE_URL value: "https://employee.mycompany.com/remoteEntry.js"
```

7.2 Shell Service

```
apiVersion: v1 kind: Service metadata: name: shell-service spec: type: ClusterIP
selector: app: shell ports: - port: 80 targetPort: 80
```

7.3 Shell Ingress

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: shell-ingress spec:
rules: - host: shell.mycompany.com http: paths: - path: / pathType: Prefix backend:
service: name: shell-service port: number: 80
```

7.4 Employee Deployment

`employee-deployment.yaml`

```
apiVersion: apps/v1 kind: Deployment metadata: name: employee spec: replicas: 3
selector: matchLabels: app: employee template: metadata: labels: app: employee spec:
containers: - name: employee image: .azurecr.io/employee: ports: - containerPort: 80
```

7.5 Employee Service

```
apiVersion: v1 kind: Service metadata: name: employee-service spec: type: ClusterIP
selector: app: employee ports: - port: 80 targetPort: 80
```

7.6 Employee Ingress

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: employee-ingress spec:
rules: - host: employee.mycompany.com http: paths: - path: / pathType: Prefix backend:
service: name: employee-service port: number: 80
```

8. Runtime Flow Diagram

User → shell.mycompany.com | | (initial load) ↓ Shell Angular App | | (navigate to /employee) ↓ loadRemoteModule() | | (HTTP request) ↓ employee.mycompany.com/remoteEntry.js | ↓ Employee Remote Module Loaded

9. CI/CD Strategy

Shell Pipeline

- Install dependencies
- `nx affected:build`
- Docker build
- Push to ACR
- Deploy to AKS

Employee Pipeline

- Same steps, independent
- Deployment does not require shell redeploy

This enables independent release cycles.

10. Key Advantages

- Independent deployments
- Lazy loading of remotes
- Zero coupling between shell and employee
- Runtime composition via Module Federation
- Scalable for large enterprise Angular platforms
- AKS provides isolation, autoscaling, and resilience

Nx Microfrontend Architecture (Shell + Employee Remote)
Production Deployment Blueprint for AKS

1. Architecture Overview

A production-grade Nx microfrontend setup using Module Federation consists of:

- Shell (Host App)
- Angular + Nx
- Module Federation Host
- Deployed as an NGINX pod on AKS
- Loads remotes at runtime via

- No remote code bundled at build time
- Employee (Remote App)
- Angular + Nx
- Module Federation Remote
- Exposes modules/components to the shell
- Built and deployed independently
- Served via its own NGINX pod on AKS
- Independent CI/CD pipelines
- Shell and Employee can be released separately
- Shell loads Employee lazily at runtime

2. Nx Workspace Structure

- and are apps (deployable).
- Shared logic lives in libs (not part of MFE boundary).
- Remotes are not libraries; they are full Angular apps.

3. Module Federation Configuration

Shell (Host)

Employee (Remote)

4. Lazy Loading the Employee Remote

Lazy loading is automatic when using .

This ensures:

- No remote code is loaded until is accessed.
- Remote is fetched via HTTP at runtime.
- Shell and Employee remain independently deployable.

5. Build Commands

Shell

Employee

Each produces:

6. Dockerfiles

Shell Dockerfile

Employee Dockerfile

7. AKS Deployment Blueprint

7.1 Shell Deployment

7.2 Shell Service

7.3 Shell Ingress

7.4 Employee Deployment

7.5 Employee Service

7.6 Employee Ingress

8. Runtime Flow Diagram

9. CI/CD Strategy

Shell Pipeline

- Install dependencies
-
- Docker build
- Push to ACR
- Deploy to AKS

Employee Pipeline

- Same steps, independent
- Deployment does not require shell redeploy

10. Key Advantages

- Independent deployments
- Zero coupling between shell and employee
- Lazy loading ensures fast shell startup
- Remotes can be versioned and rolled back independently
- Perfect for large enterprise Angular platforms